

Project Report: Containerized Web Application with PostgreSQL

Owner: shobhit panchal

Repository: <https://github.com/Shobhitpanchal/docker-assignment>

SAP: 500119775 **Subject:** Containerization & Orchestration

Project Title: Dockerized Express API with PostgreSQL

Date: March 2026

Table of Contents

1. Introduction
 2. System Architecture
 3. Backend & Dockerfile
 4. Database & Dockerfile
 5. Compose configuration & Network
 6. Persistence & Testing
 7. How to Run
 8. Conclusion
-

1. Introduction

This repository implements a minimal Express.js backend (Node.js 18) and a PostgreSQL 15 database, both containerized and orchestrated with Docker Compose. The project demonstrates multi-stage builds, non-root runtime, named volumes for persistence, and explicit Compose network configuration.

Implemented features (code inspected): - Backend (`backend/index.js`) creates a `users` table at startup and exposes endpoints: `POST /add`, `GET /users`, `GET /health`. - Backend dependencies are in `backend/package.json`: `express` and `pg`. - Backend `Dockerfile` is a multi-stage image using `node:18-alpine`, and creates a non-root user `appuser` for runtime. - Database `Dockerfile` uses `postgres:15-alpine` and sets default credentials via environment variables. - Compose (`docker-compose.yml`) defines services `db` and `backend`, a named volume `pgdata`, and an external network `mynet` with static IPv4 addresses.

2. System Architecture

Services and connectivity (as configured):

- **db** (service name **db**, container_name **postgres_db**)
 - Built from **database/Dockerfile** using **postgres:15-alpine**
 - Environment: **POSTGRES_DB=mydb**, **POSTGRES_USER=myuser**, **POSTGRES_PASSWORD=mypassword**
 - Named volume: **pgdata** → **/var/lib/postgresql/data**
 - Static IPv: **192.168.1.10** (on **mynet**)
- **backend** (service name **backend**, container_name **backend_app**)
 - Built from **backend/Dockerfile** (multi-stage)
 - Environment: **POSTGRES_HOST=db**, same DB credentials
 - Ports: **3000:3000** (host → container)
 - Static IPv: **192.168.1.11** (on **mynet**)

Communication: backend resolves Postgres at the **db** hostname (via Docker DNS) using credentials from environment variables.

Note: **docker-compose.yml** sets **networks: mynet: external: true**. Create the **mynet** network or change the Compose file to create an internal network.

3. Backend & Dockerfile

- **backend/index.js** summary:
 - Uses pg Pool with config from **POSTGRES_*** env vars (port defaults to 5432)
 - Creates **users** table on startup if missing:

```
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  name TEXT  
);
```

- Endpoints:
 - **POST /add** — expects JSON { "name": "..."} and inserts a user
 - **GET /users** — returns all users as JSON
 - **GET /health** — returns OK
- **backend/Dockerfile** notes:
 - Two-stage build using **node:18-alpine**
 - **npm install** happens in the **builder** stage
 - Runtime stage adds **appuser** and runs the app as that non-root user

Security note: avoid baking plaintext credentials into images; use Compose env or secrets.

4. Database & Dockerfile

- database/Dockerfile summary:
 - Base image: `postgres:15-alpine`
 - Sets environment variables `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD` as defaults
 - Declares `VOLUME ["/var/lib/postgresql/data"]` for persistence

Persistence is provided by the named volume `pgdata` (see below).

5. Compose configuration & Network

Key excerpts from `docker-compose.yml`:

- Service `db` uses `build: ./database`, volume `pgdata:/var/lib/postgresql/data`, and static IP `192.168.1.10` on `mynet`.
- Service `backend` uses `build: ./backend`, `depends_on db`, sets `POSTGRES_HOST=db`, maps port `3000`, and static IP `192.168.1.11` on `mynet`.
- Volumes: `pgdata`: defined at the bottom of the Compose file.
- Networks: `mynet` declared as external. If this network is missing, create it with an appropriate driver (bridge or macvlan) and subnet.

Example to create a simple bridge `mynet`:

```
docker network create --driver bridge mynet --subnet 192.168.1.0/24
```

6. Persistence & Testing

The `pgdata` named volume preserves Postgres data between container restarts.

Example verification steps:

```
docker compose up -d --build
# health
curl http://localhost:3000/health
# add user
curl -X POST http://localhost:3000/add -H "Content-Type: application/json" -d '{"name":"ali"
# list users
curl http://localhost:3000/users
docker compose down
docker compose up -d
curl http://localhost:3000/users # inserted user should still be present
```

Volume commands:

```
docker volume ls
docker volume inspect <PROJECT>_pgdata
```

Warning: `docker compose down -v` removes volumes and deletes data.

7. How to Run

1. If `mynet` is external, create it first (example above).
2. From the project root:

```
docker compose up --build -d
```

3. Test the service endpoints (examples above).

If you prefer an internal network, edit `docker-compose.yml` and remove `external: true` under `networks.mynet`.

8. Conclusion & Recommendations

This project demonstrates a concise, secure containerized stack. Recommendations to improve production-readiness:

- Use Compose secrets or an external secrets manager instead of plaintext env vars for DB credentials.
 - Add Compose healthchecks and readiness probes for better orchestration.
 - Add CI integration to run automated smoke tests against the Compose stack.
 - Replace static IP assignment with service discovery unless explicit IPs are required.
-

End of Report